

• METHODOLOGY

claude code · plan-first delivery

spec → plan →
red-team → war

Grill it into a spec, grill that into a plan, harden it against the codebase, then let a workflow of agents go to war.

• spec

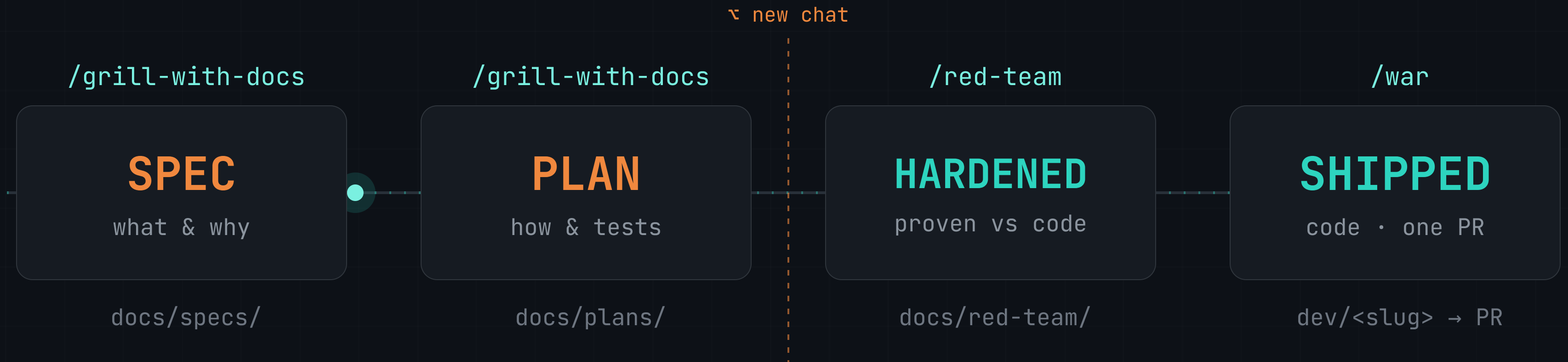
• plan

• red-team

• war

THE PIPELINE

Four commands, four artifacts



Each step writes a durable markdown artifact and hands it to the next. The plan is the handoff: a fresh chat picks it up to red-team and ship.

`spec → plan → red-team → war`

What to build, and why

01

/grill-with-docs

- docs/specs/<feature>.md



▶ The problem & the why

Requirements and acceptance criteria – what success looks like.

▶ No implementation choices

It stays decision-free on purpose. How we build it is the plan's job.

▶ Grilled until unambiguous

The agent interviews me down every branch of the design tree, one decision at a time.

How to build it

02

`/grill-with-docs <spec>`

- `docs/plans/<feature>.md`



▶ Decisions become choices

The spec's "what" is turned into concrete files, interfaces, and approach.

▶ Tests are mapped

Every change carries the test that proves it. The gate is defined here.

▶ Phase-decomposable

Written so `/war` can fan it out into phases and parallel tasks.

Cross the new-chat boundary

● planning chat

spec → plan

/grill-with-docs

/grill-with-docs <spec>

produces → docs/plans/<feature>.md



new chat

● execution chat

red-team → war

/red-team <plan>

/war <plan> --afk

consumes → the plan file

Clean context

Execution doesn't carry the planning debate.

Fresh budget

/war is a long run – start with a full window.

Reproducible

The plan file is portable – anyone can pick it up.

spec → plan → red-team → war

Harden the plan

03

/red-team <plan>

- docs/red-team/<plan>.md

✓ **CLEARED**

► Adversarial checks

A universal spine of attacks, plus probes tailored to this plan.

► Proven, not assumed

Runs the plan's tests and edits in throwaway sandboxes – never your repo.

► Grills, then patches

Every blocker resolved; the plan is rewritten in place until CLEARED.

Work • Audit • Refine

```
/war <plan> --working dev/<slug> --landing master --afk
```

01

WORK

Fresh worker agents implement each task in isolated git worktrees, writing the mapped tests.

02

AUDIT

Independent read-only auditors review the pinned SHA. Critical / Major findings block.

03

REFINE

A serial merge queue rebases, re-runs the gate, and lands approved tasks.

04

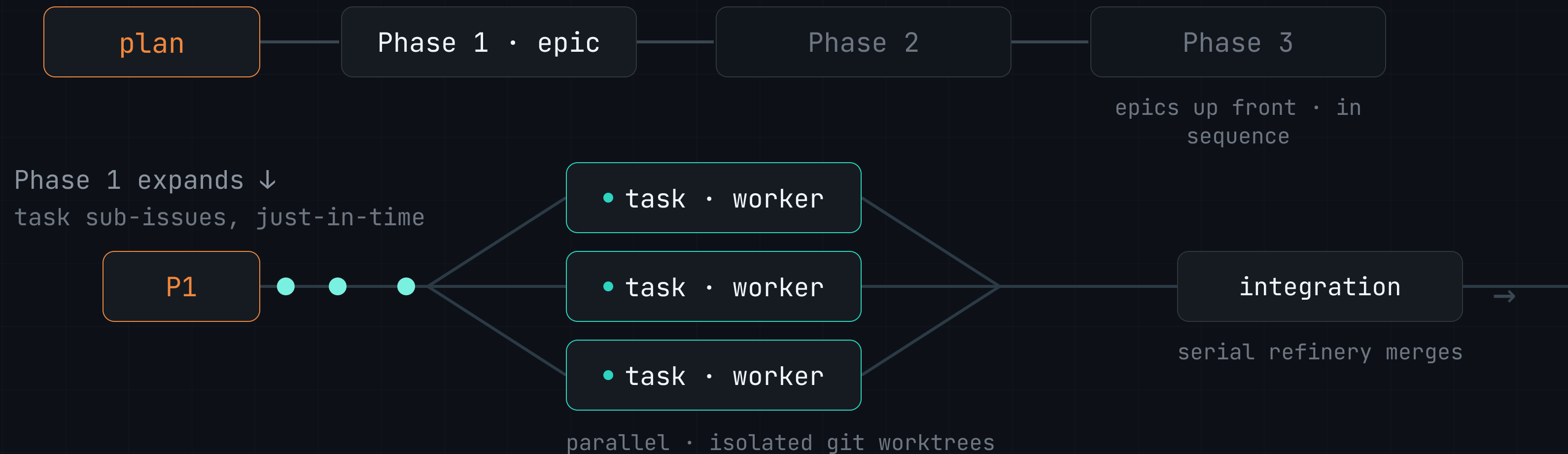
RECORD

A write-scoped servitor captures the phase's durable learnings to memory.

Repeats per phase. Each phase lands on your working branch as one `--no-ff` commit; one PR opens at the end.

INSIDE /war

Plan → phases → tasks



THE CAST

Five roles in the workflow

Lead

`your chat`

Decomposes the plan, proposes the issues, approves, and checks in at each phase.

↳ Mayor

Worker

`Agent · sonnet`

Implements one task in an isolated git worktree, writing the mapped tests.

↳ Polecat

Auditor

`Agent · opus · R0`

Read-only review of the pinned SHA. Severity-tags findings; must be unanimous.

↳ Nun

Refinery

`merge queue`

Rebases, re-runs the gate, and lands approved tasks — one at a time.

↳ Refinery

Servitor

`Agent · write-scoped`

After a phase lands, records its durable learnings to memory.

↳ bd remember

QUALITY

The audit gate



OPTIONAL · /war-room

Configure the run

/war-room

picks models & effort per role

balanced

thorough ✓

economy

.claude/war/config.json

role	model	effort
worker	opus	max
auditor	opus	high
refiner	sonnet	high
servitor	opus	default ← bumped

► Models & effort per role

Set the model and thinking effort for each seat – even put a worker on ultrathink.

► Start from a preset

balanced · thorough · economy – then only the overrides you ask for.

► Auto-discovered

Writes .claude/war/config.json; /war picks it up next run. No file → stock defaults.

MY GO-TO

/war-room thorough preset + servitor Opus
(default effort)

⚠ absolutely pumps tokens

spec → plan → red-team → war

SCALE

Three ways to run it

01 Interleaved, ad
infinitem

alternate plan / execute across parallel
chats

02 Batch of plans

make N plans, then /loop red-team → war

03 From open issues

turn the backlog into specs, then /loop

PATTERN 01 / interleaved

• parallel chats ×N

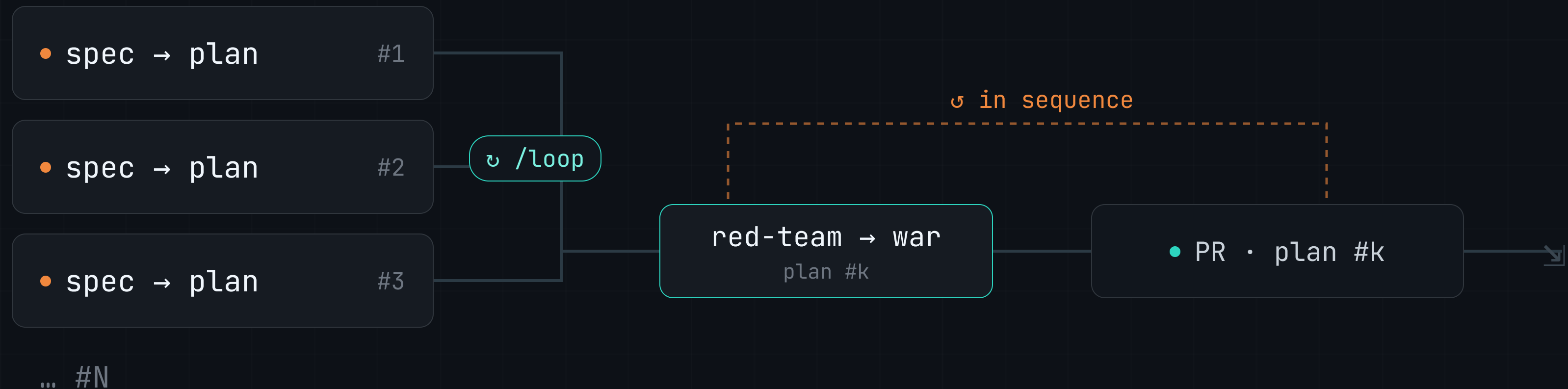
Interleaved, ad infinitum



Spec → plan, then red-team → war in a fresh chat. Repeat for the next feature and spin up more chats in parallel until the backlog is empty.

A batch of plans

MAKE N PLANS · PARALLEL

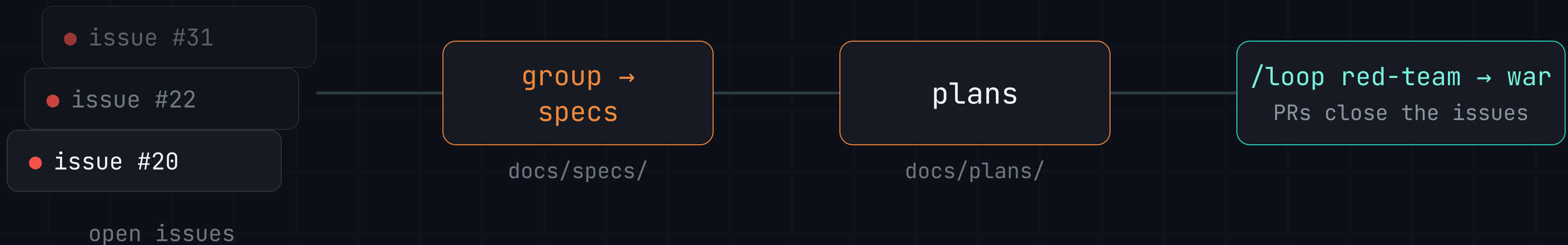


Make the whole batch of plans first, then one `/loop` runs `red-team → war` over each in sequence – a queue you start and walk away from.

`spec → plan → red-team → war`

PATTERN 03 / from issues

Straight from the backlog



Point a workflow at your open issues: group them into specs, convert to plans, then `/loop`. The PRs land and close the issues.

`spec → plan → red-team → war`

Where specs come from



THE OVERNIGHT RUN • 2026-06-25 → 26

One night. Five PRs.

```
/loop 5 plans → /red-team <plan> → /war <plan> --working dev/<slug> --landing master --afk
```

272

subagents
orchestrated

28

phases

14.1_M

tokens consumed

5

ready-to-merge PRs

✓ context < 90% of 1M

✓ zero compactions

✓ 0 Critical / Major escalated

8 follow-up issues filed

If your plans are fleshed out enough, a whole batch can land while you sleep.

Run it tonight

`/war-room`

configures the run

models & effort per role



`/red-team`

hardens the plan

proven vs the codebase



`/war`

executes it

work · audit · refine



```
/loop for each plan:  
run /red-team <plan> first, then  
/war <plan> --working dev/<slug> --landing master --afk
```

The audit gate plus your CI make this safe to run nightly. Author good plans – the workflow does the rest.